Treaps

Árvores de busca aleatorizadas

J.D.

UFABC

16 de outubro de 2026

O problema

Dicionário: manter $S \subset U$ sob busca, inserção e remoção.

O problema

Dicionário: manter $S \subset U$ sob busca, inserção e remoção.

Árvores binárias de busca (ABB): custo proporcional à *altura*. Inserções em ordem crescente $\Rightarrow$ altura $n - 1$.

O problema

Dicionário: manter $S \subset U$ sob busca, inserção e remoção.

Árvores binárias de busca (ABB): custo proporcional à *altura*. Inserções em ordem crescente $\Rightarrow$ altura $n - 1$.

Ideia: usar aleatoriedade para que a árvore se comporte como uma ABB *construída em ordem aleatória*, sem depender da ordem das operações.

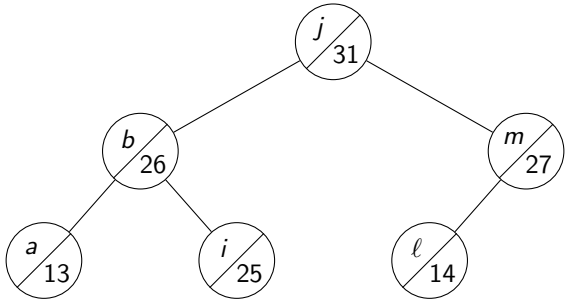
Definição

Uma **treap** é uma árvore binária em que cada nó v tem $\mathrm{chave}(v)$ e $\mathrm{prioridade}(v)$ tais que

- é **ABB** nas chaves;
- é **heap de máximo** nas prioridades: $\mathrm{prioridade}(v) >$ prioridade de todo descendente próprio.

Supomos chaves distintas e prioridades distintas.

Exemplo



letras = chaves números = prioridades em-ordem: a, b, i, j, ℓ, m

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Ideia: a heap obriga a raiz a ser o nó de maior prioridade; a busca determina quais chaves vão para cada lado; aplique indução nas subárvores.

Unicidade

Proposição

Dado um conjunto de pares (chave, prioridade) com chaves distintas e prioridades distintas, existe uma ***única*** *treap que o representa.*

Ideia: a heap obriga a raiz a ser o nó de maior prioridade; a busca determina quais chaves vão para cada lado; aplique indução nas subárvores.

Consequência: a forma depende só de chaves e prioridades, *não* da ordem de inserção.

Rotações

Consertam a *heap* sem estragar a ABB.



A ordem em-ordem A, x, B, y, C é preservada; custo $O(1)$.

Operações

Busca como em ABB; custo $\propto$ altura.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x insere como folha com prioridade **sorteada**; enquanto x tem prioridade maior que a do pai, rotaciona x para cima.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x insere como folha com prioridade **sorteada**; enquanto x tem prioridade maior que a do pai, rotaciona x para cima.

Remoção de x enquanto x não é folha, rotaciona x com o filho de **maior** prioridade; depois remove a folha.

Operações

Busca como em ABB; custo $\propto$ altura.

Inserção de x insere como folha com prioridade **sorteada**; enquanto x tem prioridade maior que a do pai, rotaciona x para cima.

Remoção de x enquanto x não é folha, rotaciona x com o filho de **maior** prioridade; depois remove a folha.

Número de rotações $\leq$ profundidade (inserção) ou altura da subárvore (remoção) de x: todas as operações custam O(altura).

Aleatoriedade

- Prioridade de cada nó: uniforme em $[0, 1]$, sorteada na inserção, independente das demais.
- **Adversário oblívio**: a sequência de operações é escolhida independentemente das prioridades.

Aleatoriedade

- Prioridade de cada nó: uniforme em $[0, 1]$, sorteada na inserção, independente das demais.
- **Adversário oblívio**: a sequência de operações é escolhida independentemente das prioridades.

Pela unicidade, após qualquer sequência oblívia de operações a árvore é **a treap do conjunto atual, com prioridades i.i.d. uniformes**.
$\Rightarrow$ basta analisar uma treap com n nós e prioridades i.i.d. uniformes.

Profundidade esperada

Seja x_k o k-ésimo menor elemento e $X(j,k) = \{x_j, \ldots, x_k\}$.

Proposição

*x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Profundidade esperada

Seja x_k o k-ésimo menor elemento e $X(j,k) = \{x_j, \ldots, x_k\}$.

Proposição

*x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Logo $\mathbb{P}[\, x_j \text{ ancestral de } x_k \,] = \dfrac{1}{|k-j|+1}$ e

$$\mathbb{E}D_k = \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} = H_k + H_{n-k+1} - 2 \leq 2 \ln n.$$

Profundidade esperada

Seja x_k o k-ésimo menor elemento e $X(j,k) = \{x_j, \ldots, x_k\}$.

Proposição

*x_j é ancestral próprio de x_k $\iff$ x_j tem a **maior prioridade** em $X(j,k)$.*

Logo $\mathbb{P}[\, x_j \text{ ancestral de } x_k \,] = \dfrac{1}{|k-j|+1}$ e

$$\mathbb{E}D_k = \sum_{j<k} \frac{1}{k-j+1} + \sum_{j>k} \frac{1}{j-k+1} = H_k + H_{n-k+1} - 2 \leq 2 \ln n.$$

Portanto busca, inserção e remoção custam $O(\log n)$ **em esperança**.

Alta probabilidade

Theorem

Numa treap com $n \geq 2$ nós e prioridades i.i.d. uniformes, com probabilidade pelo menos $1 - 2n^{-7}$ a profundidade de todo nó é menor que $2\mathrm{e}^2 \ln n < 15 \ln n$.

Alta probabilidade

Theorem

Numa treap com $n \geq 2$ nós e prioridades i.i.d. uniformes, com probabilidade pelo menos $1 - 2n^{-7}$ a profundidade de todo nó é menor que $2\mathrm{e}^2 \ln n < 15 \ln n$.

Ideia da prova

- $D_k = L_k + R_k$ (ancestrais à esquerda e à direita); cada lado é soma de Bernoullis *independentes* de parâmetros $1/i$.
- Chernoff: $\mathbb{P}\big[\, L_k \geq \mathrm{e}^2 \ln n \,\big] < n^{-8}$, e o mesmo para R_k.
- União sobre os n nós.

Resumo

- Treap = ABB nas chaves + heap de máximo nas prioridades; **forma única**.
- Inserção e remoção por **rotações**, $O(1)$ cada.
- Prioridades aleatórias $\Rightarrow$ árvore equivale a ABB de ordem aleatória.
- $\mathbb{E} D_k \leq 2 \ln n$.
- Altura $< 15 \ln n$ com probabilidade $\geq 1 - 2n^{-7}$.